

Predicting Compaction Utility from Lakehouse Table Metadata

Jannic Alexander Cutura^{1,2} Subash Prakash³

¹European Central Bank ²DSTI School of Engineering ³Nordcloud (an IBM Company)

March 2026

Outline

- 1 Motivation
- 2 Methodology
- 3 Results
- 4 Cross-Schema Generalization
- 5 Discussion
- 6 Conclusion & Future Work

The Small File Problem

Lakehouse table formats (Iceberg, Delta Lake, Hudi):

- Append-only, log-structured writes
- Each commit adds new immutable Parquet files
- Snapshot isolation + time travel

Consequence: progressive file accumulation

- Query planning scans every file's metadata
- S3 LIST/GET calls are priced per request
- Excessive file-open overhead degrades scans

At Scale

Catalogs with thousands of tables accumulate **millions** of undersized files.

AutoComp (LinkedIn): 35,000+ Iceberg tables.

Compaction

Rewrites many small files → fewer target-sized files.

But compaction **costs compute**.

When is it worth it?

The Gap

- All open-source scheduling is **threshold-driven**:
 - Iceberg: purely imperative (no built-in scheduler)
 - Delta Lake: triggers on small-file counts
 - Hudi: commit-count / elapsed-time rules
- AutoComp uses hand-crafted heuristics + Pareto ranking
- Databricks Predictive Optimization claims ML, but is proprietary
- **No open study** examines *which* metadata features determine compaction utility

Our Goal

Predict compaction utility from table metadata alone, using ML, and identify which features matter.

Problem Formulation

File count reduction ratio:

$$r_{\text{fc}} = 1 - \frac{n_{\text{after}}}{n_{\text{before}}}$$

Two tasks:

- 1 **Classification:** Does the table need compaction? ($r_{\text{fc}} > 0.5$)
- 2 **Regression:** Predict the continuous $r_{\text{fc}} \in [0, 1]$ for cost-aware scheduling

Key constraint: features extracted only from Iceberg metadata (manifests, snapshots, partition stats).
No data files read.

Parameter grid: 7 axes, 2,376 combinations

- Row counts: 10^4 to 10^6
- File-size targets: 8 KB to 128 MB
- Partitions: 1 to 100
- Writers: 1 or 5
- Batches: 1 to 20
- Skew: uniform vs. Zipfian
- Payload: 10 or 50 UUID columns

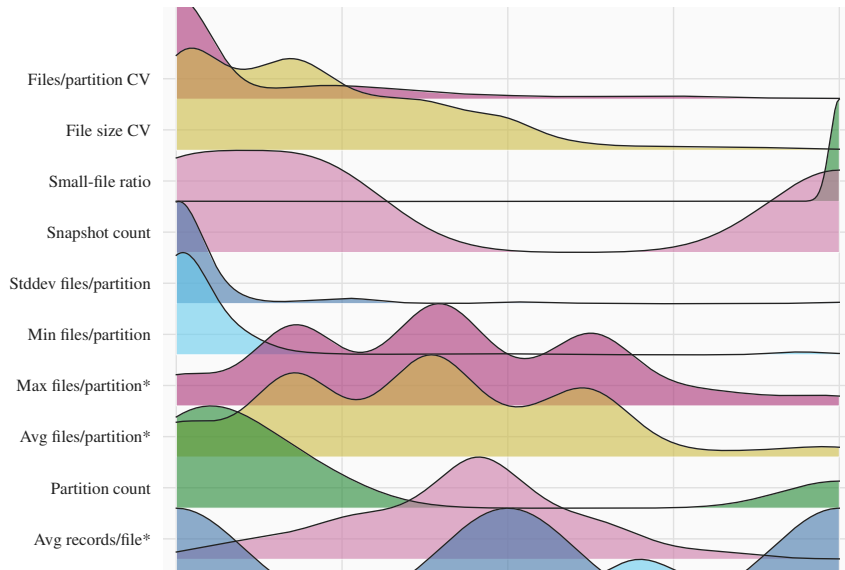
Heterogeneous writes:

- Batch 1: 50% of rows (initial load)
- Even batches: $8\times$ smaller file target
- Breaks the “tautology trap”

Compaction: Iceberg bin-packing

- Target: 128 MB
- Min: 96 MB (75%)
- Max: 230 MB (180%)

17 Metadata Features



Classification: Binary Decision

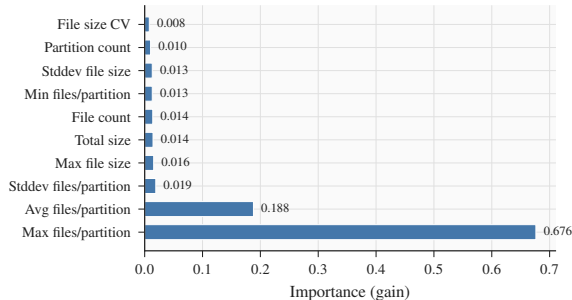
Classification (held-out test, $n = 476$)

Model	Acc.	F1
XGBoost	1.000	1.000
Logistic Reg.	0.815	0.885
Threshold ($k=4$)	1.000	1.000

Single-feature threshold perfectly solves

classification:

`max_files_per_partition > 4`



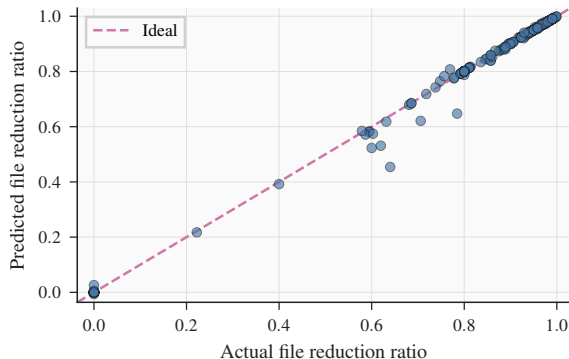
Two features account for **86.4%** of classifier gain.

Regression: Continuous Prediction

Regression (held-out test)

Model	RMSE	R^2
XGBoost	0.013	0.998
OLS	0.236	0.361

- OLS cannot capture the nonlinear saturation curve
- Two features account for **95.2%** of regressor gain
- Spearman $\rho = 0.965$: monotonic but not linear



Predicted vs. actual r_{fc} on the held-out test set.

TPC-H Validation (No Retraining)

96 Iceberg tables from 6 TPC-H schemas (lineitem, orders, customer, part, partsupp, supplier)

- Realistic data types (strings, dates, decimals)
- 7–16 columns per schema
- Scale factor ~ 10 (468.8 M rows, 15 GB)

Cross-schema results

Task	Acc./RMSE	F1/ R^2
Clf (XGBoost)	0.979	0.987
Clf (Threshold)	0.833	0.908
Reg (XGBoost)	0.058	0.976

Key findings:

- 97.9% accuracy, zero false positives
- +14.6 pp over threshold baseline
- In a 1,000-table catalog: threshold triggers ~ 167 unnecessary jobs; ML triggers **none**

Table: *

Per-schema breakdown

Schema	Clf Acc.	Reg R^2
lineitem	1.000	0.997
orders	1.000	1.000
part	1.000	1.000
supplier	1.000	1.000
customer	0.938	0.874
partsupp	0.938	0.867

Key Insights

Tautology Trap

Benchmarks using only small synthetic files produce misleadingly perfect metrics. File-size heterogeneity is essential for non-trivial prediction targets.

Feature Dominance

Partition-level file counts (`max_files_per_partition`, `avg_files_per_partition`) dominate both tasks.

- Binary decision: fully solvable by a single threshold
- Continuous prediction: requires nonlinear modelling

Practical Implication

AutoComp's compaction traits already include these features. Our results confirm they are sufficient for binary scheduling, but **cost-aware scheduling** requires a learned model.

Threats to Validity

- Synthetic UUID payloads compress differently from real columnar data
- Partition structure is generated, not domain-derived
- Query patterns are not modelled
- Spark-local execution $\neq$ distributed clusters with S3
- 89.3% positive class prevalence may inflate accuracy
- TPC-H partially mitigates synthetic-data concerns, but production tables with schema evolution, delete files, and mixed CoW/MoR remain untested

Conclusion

- 1 **Binary compaction decision** is trivially solvable by a single partition-level threshold
- 2 **Continuous reduction ratio** requires nonlinear modelling
(XGBoost $R^2 = 0.998$ vs. OLS $R^2 = 0.361$)
- 3 **Models generalize** across schemas: 97.9% accuracy on TPC-H with zero false positives
- 4 Compaction utility is determined by **structural metadata**, not data content

Future work:

- Extend to Delta Lake and Hudi
- Add query-workload features and cloud-cost metrics
- Multi-table scheduling via contextual bandits
- Validation on production catalogs

All code and data publicly available.

Thank You

Questions?

Jannic Alexander Cutura | Subash Prakash

`jannic.cutura@dsti.institute` | `subash.prakash@nordcloud.com`